Cloud Posture Dashboard: Design Rationale

Problem Statement & User Persona

- **User Persona:** The Cloud Security Architect. They manage dozens of cloud accounts across AWS, Azure, and GCP and are frustrated by having to log into multiple different consoles to check for basic security flaws.
- **Problem Statement:** Security teams lack centralized visibility. They need a unified dashboard that normalizes misconfiguration data (like public storage buckets or unencrypted databases) across disparate cloud providers. The goal is to allow them to instantly filter by severity and cloud provider so they can prioritize what needs to be fixed today.

Proposed Features & Prioritization To deliver a pragmatic solution, I prioritized features based on the core need for cross-cloud visibility:

- **P0 (Must Have): Unified Filter Sidebar & Data Table.** A robust sidebar filter next to a standard data table. *Reasoning: The core problem is finding specific issues across different environments. A standardized table is the most efficient way to achieve this.*
- **P1 (Should Have): Actionable Remediation Drawer.** A slide-out panel showing exactly how to fix the misconfiguration. *Reasoning: Identifying a problem is only half the battle; providing the fix (like a CLI command) reduces friction between Security and DevOps.*
- **P2 (Could Have): Global Posture Score.** A top-level grade/score widget. *Reasoning: Great for executive reporting, but less critical for the day-to-day engineer doing the actual work.*

Success Metrics To measure the effectiveness of this dashboard, we will track:

1. **Filter Engagement Rate:** How frequently users are utilizing the cross-cloud and severity filters.
2. **Time-to-Discover:** The time it takes a user from logging in to locating a specific flagged resource.
3. **Remediation Action Clicks:** Tracking how often users copy the remediation code snippets from the details drawer.

Development Action Items (Discussion Points for Engineering)

1. **Data Normalization:** AWS, Azure, and GCP all use different terminology (e.g., EC2 vs. Virtual Machine). We need to map these to a standard schema so our UI filters work across all three simultaneously.
2. **Pagination & Performance:** A large enterprise might have 100,000+ misconfigurations. We need server-side pagination and a caching strategy so the UI table doesn't freeze.